

Bài tập: Cho tập dữ liệu về bệnh ung thư: <https://www.kaggle.com/uciml/breast-cancer-wisconsin-data>

Trong đó là các thông tin về đặc điểm của tế bào đã được ghi nhận thành các thuộc tính (radius_mean, texture_mean, ...), thể hiện dưới dạng bảng. Cùng với đó là các chẩn đoán (diagnosis) liệu tế bào đó có phải là tế bào ung thư hay không.

Hãy xây dựng mô hình Naive Bayes để dự đoán liệu một tế bào với các đặc điểm cho trước có phải là một tế bào ung thư hay không?

- Import các thư viện cần thiết và load dữ liệu từ file

```
In [1]: #Import các thư viện cần thiết
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: #Load data
data = pd.read_csv("cancer_data.csv")
data.head() #Quan sát dữ liệu cho thấy, tập dữ liệu đã được xử lí khá "sạch"
```

```
Out[2]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns



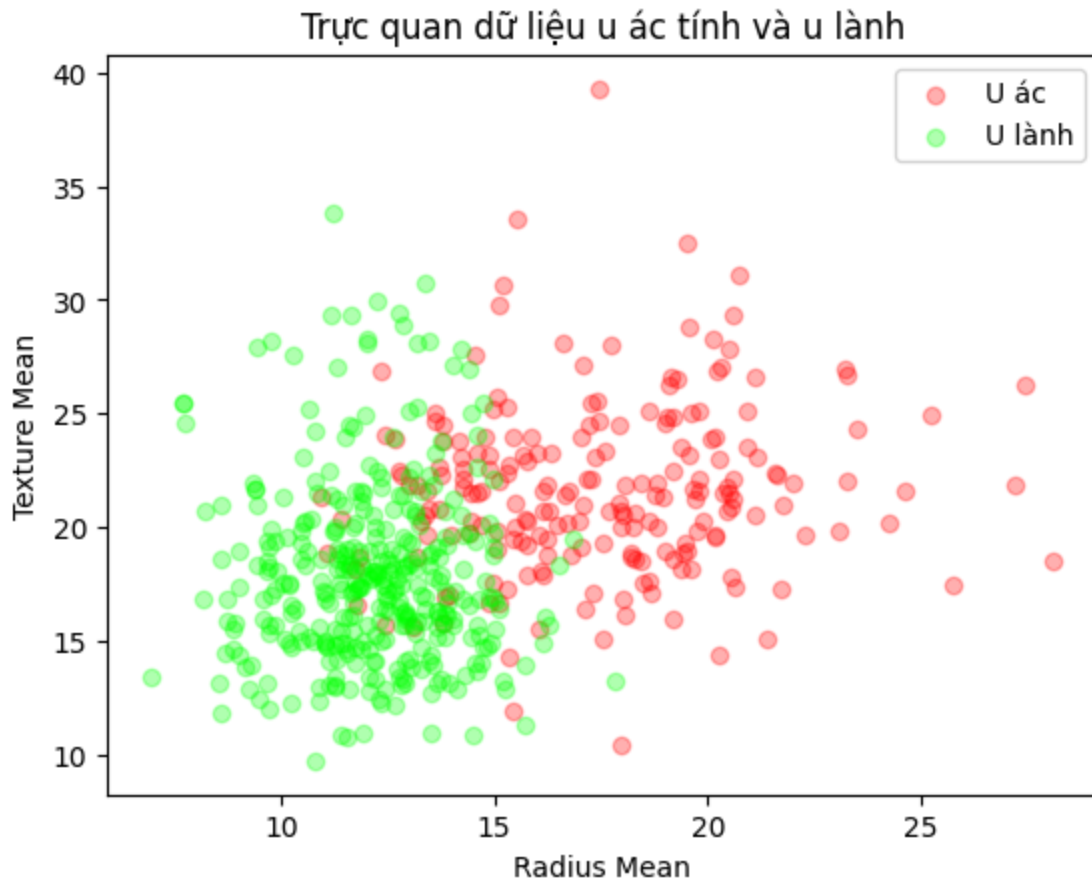
```
In [3]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    569 non-null    int64
1   diagnosis                            569 non-null    object
2   radius_mean                          569 non-null    float64
3   texture_mean                         569 non-null    float64
4   perimeter_mean                       569 non-null    float64
5   area_mean                           569 non-null    float64
6   smoothness_mean                     569 non-null    float64
7   compactness_mean                    569 non-null    float64
8   concavity_mean                      569 non-null    float64
9   concave points_mean                 569 non-null    float64
10  symmetry_mean                       569 non-null    float64
11  fractal_dimension_mean              569 non-null    float64
12  radius_se                           569 non-null    float64
13  texture_se                           569 non-null    float64
14  perimeter_se                         569 non-null    float64
15  area_se                             569 non-null    float64
16  smoothness_se                       569 non-null    float64
17  compactness_se                      569 non-null    float64
18  concavity_se                        569 non-null    float64
19  concave points_se                   569 non-null    float64
20  symmetry_se                         569 non-null    float64
21  fractal_dimension_se                569 non-null    float64
22  radius_worst                        569 non-null    float64
23  texture_worst                       569 non-null    float64
24  perimeter_worst                     569 non-null    float64
25  area_worst                          569 non-null    float64
26  smoothness_worst                    569 non-null    float64
27  compactness_worst                   569 non-null    float64
28  concavity_worst                     569 non-null    float64
29  concave points_worst                569 non-null    float64
30  symmetry_worst                      569 non-null    float64
31  fractal_dimension_worst             569 non-null    float64
32  Unnamed: 32                         0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

```
In [4]: # Loại trường không cần thiết: id, unnamed
data = data.drop(["id", "Unnamed: 32"], axis = 1)
```

```
In [5]: #Trực quan hóa về dữ liệu: u ác tính và u lành:
M = data[data.diagnosis == "M"] # dữ liệu ứng với u ác tính
B = data[data.diagnosis == "B"] # dữ liệu ứng với u lành

plt.title("Trực quan dữ liệu u ác tính và u lành")
plt.xlabel("Radius Mean")
plt.ylabel("Texture Mean")
plt.scatter(M.radius_mean, M.texture_mean, color = "red", label = "U ác", alpha = 0.5)
plt.scatter(B.radius_mean, B.texture_mean, color = "lime", label = "U lành", alpha = 0.5)
plt.legend()
plt.show()
```



- Tiền xử lí trước khi huấn luyện mô hình:

```
In [6]: # Câu hỏi: Đưa trường chẩn đoán từ dạng chữ thành dạng số
# Code #####
data.diagnosis = [1 if i == "M" else 0 for i in data.diagnosis]
data.head()
#####
```

```
Out[6]:
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	1	17.99	10.38	122.80	1001.0	0.11840
1	1	20.57	17.77	132.90	1326.0	0.08474
2	1	19.69	21.25	130.00	1203.0	0.10960
3	1	11.42	20.38	77.58	386.1	0.14250
4	1	20.29	14.34	135.10	1297.0	0.10030

5 rows × 31 columns

```
In [7]: # Câu hỏi: Tách thuộc tính và nhãn:
# Code #####
```

```
X = data.drop(["diagnosis"], axis=1)
y = data.diagnosis.values
X
```

```
#####
```

Out[7]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
0	17.99	10.38	122.80	1001.0	0.11840	0.26634
1	20.57	17.77	132.90	1326.0	0.08474	0.19763
2	19.69	21.25	130.00	1203.0	0.10960	0.28394
3	11.42	20.38	77.58	386.1	0.14250	0.42021
4	20.29	14.34	135.10	1297.0	0.10030	0.18707
...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.27303
565	20.13	28.25	131.20	1261.0	0.09780	0.25918
566	16.60	28.08	108.30	858.1	0.08455	0.16343
567	20.60	29.33	140.10	1265.0	0.11780	0.27412
568	7.76	24.54	47.92	181.0	0.05263	0.16169

569 rows × 30 columns



```
In [8]: # Chuẩn hóa các trường của X
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
sc.fit(X)
```

Out[8]:

▼ StandardScaler ⓘ ?

► Parameters

```
In [9]: # Câu hỏi: Chia dữ liệu train/test:
from sklearn.model_selection import train_test_split
# Code #####
x_train, x_test, y_train, y_test = \
    train_test_split(X, y, test_size = 0.3, random_state = 4)
#####
```

- Xây dựng và đánh giá mô hình Naive Bayes:

```
In [10]: # Câu hỏi: Multinomial NB:
```

```
# Code #####
from sklearn.naive_bayes import MultinomialNB
model_MNB = MultinomialNB()
model_MNB.fit(x_train, y_train)

print("Multinomial Naive Bayes score: ", model_MNB.score(x_test, y_test))

#####
```

Multinomial Naive Bayes score: 0.8713450292397661

In [11]: # Câu hỏi: Gaussian NB:

```
# Code #####
from sklearn.naive_bayes import GaussianNB
model_GNB = GaussianNB(var_smoothing=1e-3)
model_GNB.fit(x_train, y_train)

print("Gaussian Naive Bayes score: ", model_GNB.score(x_test, y_test))

#####
```

Gaussian Naive Bayes score: 0.8947368421052632

In [12]: # Câu hỏi: Dùng GNB để dự đoán thử một sample = phần tử đầu tiên của x_test:

```
# Code #####3

X_test = x_test.reset_index(drop=True) #Đánh lại chỉ số để dễ gọi

sample = X_test.iloc[0:1]

print('Label: ', y_test[0])
print('Sample: \n', sample)
print('-----\n Dự đoán nhãn: ', end='')
model_GNB.predict(sample)[0]

#####
```

Label: 0

Sample:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	\
0	14.42	16.54	94.15	641.2	0.09751	

	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	\
0	0.1139	0.08007	0.04223	0.1912	

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst	\
0	0.06412	...	16.67	21.51	111.4	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	862.1	0.1294	0.3371	0.3755	

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	0.1414	0.3053	0.08764

[1 rows x 30 columns]

Dự đoán nhãn:

Out[12]: np.int64(0)